

Sistema IoT Industrial Escalable y de Bajo Costo para la Monitorización de Tiempos de Actividad en Maquinaria Industrial: Arquitectura de Servidor, Motor de Actividades e Interfaz de Gestión

1st Hernán F. Kiesel
Universidad Nacional de Misiones
Facultad de Ingeniería
Oberá, Misiones, Argentina
hernankiesel@gmail.com

2nd Germán A. Xander
Universidad Nacional de Misiones
Facultad de Ingeniería
Oberá, Misiones, Argentina
german.xander@fio.unam.edu.ar

3rd Luis A. Urbani
Universidad Nacional de Misiones
Facultad de Ingeniería
Oberá, Misiones, Argentina
ctt.urban@gmail.com

4th Alejandro G. Maxit
Universidad Nacional de Misiones
Facultad de Ingeniería
Oberá, Misiones, Argentina
alejandro.maxit@fio.unam.edu.ar

Resumen—En la Industria 4.0, la digitalización de plantas industriales tradicionales representa un desafío crítico debido a los elevados costos de las soluciones SCADA y MES comerciales. Este artículo presenta el diseño e implementación de un sistema de Internet de las Cosas (IoT) end-to-end, caracterizado por ser de bajo costo, fácil instalación, alta usabilidad y escalabilidad horizontal. El sistema captura eventos operativos mediante desacoplamiento optoelectrónico no invasivo en las señales de contactores de 24V de maquinaria electromecánica (torno y fresadora) procesados por microcontroladores ESP32. La arquitectura de servidor, orquestada mediante Docker, integra un broker MQTT Mosquitto, un motor de ingesta de datos, una base de datos relacional MariaDB, una API REST en FastAPI, un bot de Telegram 2.0 para interacción operativa en planta y un panel Web Admin SPA en HTML5/Bootstrap 5. Se incorpora un motor de actividades automatizado que consolida sesiones operativas, calcula el desgaste de herramientas y genera reportes de KPIs. La solución demuestra que es posible implementar telemetría industrial avanzada con una inversión mínima en hardware y software de código abierto.

Index Terms—IoT Industrial, ESP32, Arquitectura de Servidor, FastAPI, Docker, MQTT, Telegram Bot, Web Admin SPA, Industria 4.0.

I. INTRODUCCIÓN Y PLANTEAMIENTO DEL PROBLEMA

En las pequeñas y medianas empresas (PyMEs) del sector metalúrgico, la eficiencia productiva depende de la maximización del tiempo de uso de maquinaria crítica como tornos y fresadoras. Sin embargo, la gestión tradicional de la producción suele basarse en la supervisión visual directa o en registros manuales en papel, métodos propensos a errores, vacíos de información y retrasos en la toma de decisiones estratégicas.

La mayoría de los equipos instalados en estas plantas corresponden a maquinaria convencional cuyos sistemas de control son electromecánicos puros, basados en contactores y

relés operados a 24V, sin interfaces digitales de comunicación ni capacidad nativa para conectarse a redes de datos. La adopción de sistemas SCADA o plataformas de ejecución de la fabricación (MES) propietarias suele ser inviable para este segmento industrial debido al costo prohibitivo del hardware industrial dedicado, licencias de software y la complejidad de instalación.

Para resolver esta problemática, el presente trabajo establece como objetivo el desarrollo de un sistema IoT integral diseñado bajo cuatro pilares fundamentales:

1. **Bajo Costo:** Utilización de hardware de desarrollo estándar (ESP32) y componentes electrónicos discretos, junto con un stack de software 100 % de código abierto (Open Source).
2. **Fácil Instalación:** Implementación de un acondicionamiento de señal no invasivo mediante optoacopladores que toman señal de los contactores de 24V sin alterar la lógica electromecánica de seguridad original de las máquinas.
3. **Fácil Uso:** Provisión de interfaces duales diseñadas según el perfil de usuario: un Bot de Telegram interactivo para operarios en planta y un panel Web Admin SPA (Single Page Application) accesible desde cualquier navegador para los supervisores.
4. **Alta Escalabilidad:** Microservicios desacoplados e independizados en contenedores Docker, lo que permite añadir nuevas máquinas, sensores o módulos de análisis sin modificar la estructura del servidor central.

II. DESCRIPCIÓN DEL SENSOR Y ACONDICIONAMIENTO HARDWARE

El segmento sensor (Edge Hardware) tiene la función de detectar en tiempo real el estado operativo del motor principal de cada máquina industrial. En el torno, el control consta de pulsadores de arranque y parada que energizan la bobina del contactor principal. En la fresadora, el encendido se realiza mediante un selector rotativo de dos posiciones de giro (horario y antihorario).

Para garantizar un aislamiento galvánico absoluto entre los circuitos de potencia industrial (24V) y la electrónica sensible del microcontrolador (3.3V), se descartó la conexión directa y se diseñó una etapa de acondicionamiento de señal basada en optoacopladores. Cuando el contactor se activa, la señal de 24V enciende el diodo emisor del optoacoplador, cuyo fototransistor conmuta una entrada digital lógica limpia (GPIO) en el microcontrolador ESP32.

Cada máquina dispone de un módulo ESP32 independiente. El firmware integrado ejecuta una lectura continua con filtro anti-rebote (debounce), detectando cambios de estado e interrupciones de parada de emergencia. Al producirse un evento, el dispositivo emite inmediatamente un mensaje estructurado vía Wi-Fi utilizando el protocolo ligero MQTT (Message Queuing Telemetry Transport).

III. ARQUITECTURA DE SERVIDOR E INFRAESTRUCTURA

La capa de servidor constituye el núcleo del sistema, encargada de la ingesta de telemetría, almacenamiento persistente, lógica de negocio automatizada y suministro de interfaces de usuario. Toda la infraestructura se orquesta mediante contenedores independientes gestionados por Docker Compose, garantizando un despliegue repetible en cualquier servidor central o dispositivo de cómputo en la red local.

III-A. Microservicios Orquestados

La arquitectura del servidor se compone de ocho microservicios desacoplados:

- **SWAG (Nginx Reverse Proxy):** Punto de entrada único que provee cifrado SSL/TLS (HTTPS) y enrutamiento seguro hacia los servicios internos.
- **Mosquitto (MQTT Broker):** Broker de mensajería ligero de alta velocidad en los puertos 1883 y 8883.
- **Cliente MQTT Ingesta (Python Service):** Proceso en segundo plano suscrito a los tópicos MQTT que procesa los eventos entrantes y los registra en la base de datos.
- **MariaDB:** Motor de base de datos relacional optimizado para almacenamiento persistente de eventos, actividades y métricas.
- **FastAPI REST Backend (API IoT):** Servidor de aplicaciones de alto rendimiento que expone endpoints REST, sirve la SPA del Administrador y gestiona los algoritmos de ajuste de datos.
- **Bot de Telegram 2.0:** Servicio conversacional para operarios y administradores en planta.
- **Grafana:** Motor de tableros de control para análisis visual interactivo e historial de producción.

- **phpMyAdmin:** Interfaz de administración web para mantenimiento de base de datos.

III-B. Motor de Actividades Automatizado

Un componente innovador del servidor es el **Motor de Actividades Automatizado**. A diferencia de los sistemas tradicionales que se limitan a guardar eventos aislados de encendido y apagado, este motor consolida la telemetría en sesiones continuas de trabajo (actividades) vinculadas al operario de turno y a la herramienta instalada.

La lógica de consolidación funciona de la siguiente manera:

1. **Inicio de Sesión:** Al recibir un evento de encendido (ACTIVA), el sistema busca si la máquina cuenta con un operario y herramienta asignados en la tabla de asignaciones actuales. Si existe una actividad en curso para esa máquina, la continúa; si no, genera una nueva actividad en estado EN_CURSO.
2. **Cierre por Tiempo de Inactividad ($T_{\text{inactividad}}$):** Cuando la máquina entra en estado PARADA, se inicia un temporizador de inactividad configurable (por defecto 10 minutos). Si la máquina vuelve a arrancar antes del tiempo límite, la actividad continúa activa acumulando tiempo de trabajo efectivo. Si se supera el límite $T_{\text{inactividad}}$, la actividad se da por concluida, estableciendo su estado en FINALIZADA y calculando el tiempo activo exacto consumido.
3. **Parada de Emergencia:** Si se detecta el evento PARADA_EMERGENCIA, la actividad se cierra inmediatamente bajo ese estado y se dispara una alerta prioritaria de seguridad a los administradores.

IV. ESQUEMA RELACIONAL DE BASE DE DATOS

El almacenamiento de información en MariaDB se estructura mediante un modelo relacional normalizado diseñado para garantizar la integridad referencial y permitir consultas analíticas rápidas de rendimiento. El esquema completo se compone de ocho tablas descritas en la Tabla I.

V. CAPA DE GESTIÓN: WEB ADMIN SPA Y BOT DE TELEGRAM 2.0

Para satisfacer las necesidades del personal de planta y de la gerencia sin requerir software especializado instalado en cliente, el sistema provee dos interfaces de usuario independientes.

V-A. Panel Web Admin SPA (FastAPI + Bootstrap 5)

El panel de administración web está construido como una aplicación de página única (SPA) desarrollada en HTML5 Vanilla JS y CSS con Bootstrap 5, servida directamente por el backend de FastAPI en la ruta /admin. El diseño utiliza un tema visual oscuro de estilo industrial. Sus módulos incluyen:

- **Dashboard en Tiempo Real:** Tarjetas de monitoreo visual con actualización automática cada 5 segundos que muestran el estado operativo (ACTIVA, PARADA, PARADA_EMERGENCIA), el operario responsable y la herramienta montada.

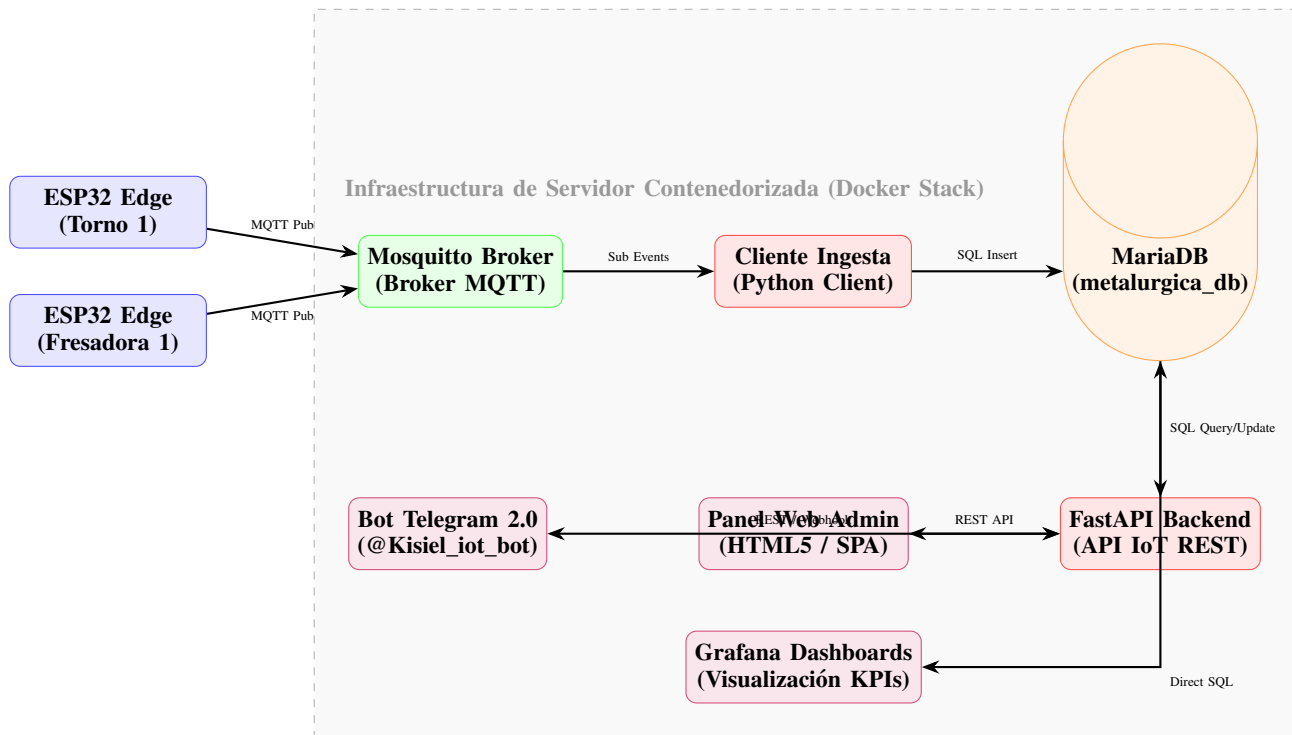


Figura 1. Diagrama de bloques de la arquitectura global del sistema IoT industrial end-to-end.

Tabla I
ESTRUCTURA DE LAS TABLAS DE LA BASE DE DATOS RELACIONAL (METALURGICA_DB)

Tabla	Claves	Campos y Descripción del Rol en el Sistema
maquinas	PK: id	nombre, tipo, ubicacion. Registro central de activos industriales.
usuarios	PK: id, UQ: telegram_id	nombre, username, password_hash, rol (ADMIN/OPERARIO), vinculación Telegram.
herramientas	PK: id, FK: maquina_id	nombre, horas_uso, horas_expectativa. Control dinámico de vida útil.
asignaciones_actuales	PK/FK: maquina_id	FK: operario_id, FK: herramienta_id. Estado de asignación en tiempo real.
registro_actividad	PK: id, FK: maquina_id	estado, codigo_estado, causa, timestamp. Registro histórico de actividades.
actividades	PK: id, FKs: maquina, operario, herramienta	fecha_inicio, fecha_fin, tiempo_activo_segundos, estado. Registro de actividades consolidadas de trabajo.
kpi_turnos	PK: id, FK: maquina_id	fecha, turno, tiempo_activo_seg, disponibilidad_pct. Indicadores de rendimiento por turno.
configuracion_sistema	PK: clave	valor, descripcion. Parámetros globales ($T_{inactividad}$, horarios de trabajo).

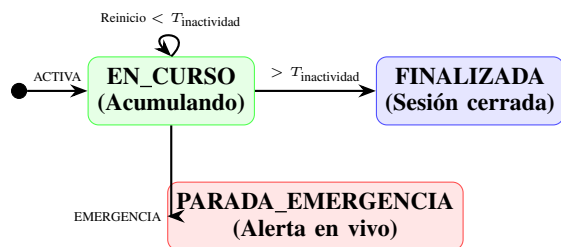


Figura 2. Diagrama de estados del Motor de Actividades Automatizado.

- **Gestión de Operarios:** Alta, baja lógica y visualización de estadísticas individuales de uso de maquinaria mediante gráficos dinámicos de barras por turno (Matplotlib).
- **Herramientas y Vida Útil:** Barras de progreso de desgaste dinámicas codificadas por colores (Verde < 70 %, Amarillo 70 – 90 %, Rojo > 90 %) según las horas de uso reales acumuladas frente a la expectativa de vida útil.
- **Control y Ajuste Retroactivo de Actividades (Fit):** Herramienta que permite al administrador realizar correcciones de datos:

- **Merge (Fusión):** Combina dos o más actividades consecutivas erróneamente fragmentadas en un único registro consolidando los tiempos activos.
- **Split (División):** Divide una actividad extensa en dos fragmentos mediante la especificación de una fecha/hora de corte.
- **Asignación Retroactiva:** Asigna operarios a sesiones registradas previamente sin operario.
- **Informes y KPIs Visuales:** Resúmenes por turno y gráficos estadísticos generados dinámicamente con Matplotlib en tiempo real, junto con la opción de exportar el historial completo en formato CSV.
- **Configuración Global:** Ajuste del parámetro $T_{\text{inactividad}}$ y horarios de inicio de turnos de trabajo.

V-B. Bot de Telegram 2.0 (@Kisiel_bot)

La interfaz conversacional operada mediante la API de Telegram actúa como la principal herramienta de interacción en la nave de producción. Sus características clave son:

- **Autenticación Basada en Roles:** Inicio de sesión mediante comando `/login` con clave cifrada. Los usuarios con rol ADMIN acceden a la gestión de operarios, configuración y reportes; los operarios con rol OPERARIO acceden al fichaje de máquinas y montaje de herramientas.
- **Fichaje de Operario y Herramienta:** El operario selecciona la máquina a operar y la herramienta a utilizar directamente mediante teclados de botones en pantalla (Inline Keyboards).
- **Diseño Limpio Sin Emojis:** Interfaz textual optimizada basada en etiquetas claras ([Gestión Operarios], [Estado Máquinas], [Cerrar Sesión]), garantizando legibilidad en cualquier dispositivo industrial.
- **Notificación Inmediata de Emergencias:** Ante una parada de emergencia generada en planta, el backend dispara un mensaje de alerta prioritaria a los administradores registrados vía Telegram.

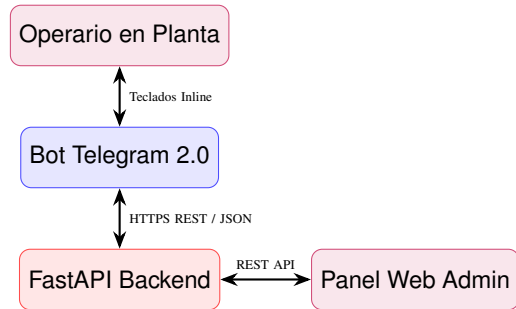


Figura 3. Flujo de interacción entre las interfaces de usuario y el servidor central.

VI. EVALUACIÓN DE RESULTADOS, ESCALABILIDAD Y COSTOS

Para validar el rendimiento y la flexibilidad del sistema, se desplegó un escenario de prueba en planta acumulando 30

días de funcionamiento continuo con 134 sesiones de actividad generadas en torno y fresadora.

VI-A. Análisis de Costos

Frente a soluciones comerciales SCADA cuyo costo por máquina supera frecuentemente los 1.500 USD en hardware más licencias periódicas, la solución desarrollada requiere una inversión en componentes hardware inferior a 45 USD por máquina (ESP32, fuentes de alimentación, circuito optoacoplador y caja estanca industrial). El software servidor opera sobre licencias de código abierto sin costos recurrentes.

VI-B. Facilidad de Despliegue y Escalabilidad

El despliegue de toda la pila de software en un nuevo servidor se ejecuta mediante un único comando (`docker compose up -d`). La adición de una nueva máquina requiere únicamente conectar un nuevo módulo ESP32 a la red Wi-Fi y registrar la máquina en la base de datos a través del panel Web Admin. La arquitectura pub-sub de MQTT garantiza que el broker pueda gestionar cientos de mensajes por segundo sin saturar el procesamiento del backend.

VII. CONCLUSIÓN

El desarrollo presentado demuestra la viabilidad técnica y económica de digitalizar maquinaria industrial convencional mediante una arquitectura IoT moderna, robusta y escalable. La integración de captadores optoelectrónicos no invasivos en el borde con un stack de servidor contenedorizado en Docker permite transformar datos brutos de contactores en indicadores clave de producción (KPIs), tiempos efectivos de trabajo y seguimiento de vida útil de herramientas.

Las interfaces duales (Panel Web Admin SPA y Bot de Telegram 2.0) ofrecen una experiencia de usuario adaptada tanto al personal de gerencia como a los operarios en planta, facilitando la adopción tecnológica y resolviendo el problema de la falta de visibilidad operativa a una fracción del costo de los sistemas comerciales.

REFERENCIAS

- [1] Espressif Systems, "ESP32 Technical Reference Manual," v4.8, 2023. [En línea]. Disponible en: <https://www.espressif.com>
- [2] OASIS Standard, "MQTT Version 5.0," 2019. [En línea]. Disponible en: <https://mqtt.org>
- [3] Docker Inc., "Enterprise Application Container Platform," 2024. [En línea]. Disponible en: <https://www.docker.com>
- [4] S. Ramírez, "FastAPI Framework for Python," 2024. [En línea]. Disponible en: <https://fastapi.tiangolo.com>
- [5] MariaDB Foundation, "MariaDB Knowledge Base," 2024. [En línea]. Disponible en: <https://mariadb.org>
- [6] Telegram Messenger Inc., "Telegram Bot API Documentation," 2024. [En línea]. Disponible en: <https://core.telegram.org/bots/api>